

Bug 报告：admin_stop 根因排查与修复方案

项目: pkudsa.airacer
日期: 2026-06-05
严重级别: P1 Critical
排查人员: Claude Code

1. 问题摘要

测试赛大量被 admin_stop 终止，282/1817（15.5%）的录制在 1 帧（0.032s）处结束，车辆从未开始移动。受影响最严重的团队 A10089 连续 12 次测试全部失败。

2. 根因确认：竞态条件

2.1 问题机制

admin_stop 的根本原因是 **Webots 启动与 STOP 文件写入之间的竞态条件**：

```
POST /api/admin/zones/{zone_id}/stop-race      (server/blueprints/admin.py:585)
→ simnode_client.cancel_race(session_id)      (server/utils/simnode_client.py:86)
→ POST /race/{race_id}/cancel                  (simnode/server.py:137)
  → RaceManager.cancel_race()                  (simnode/race_manager.py:141)
    → RaceRunner.graceful_stop()               (simnode/race_runner.py:229)
      → 写入 STOP 文件（L235）                 ← 原子操作，立即完成
      → supervisor 检测到 STOP（L542）         ← 第 1 帧就检测到
```

关键时序： _launch_webots() 启动 Webots 子进程后，世界加载需要时间。但如果在此期间 graceful_stop() 被调用，STOP 文件已存在，supervisor 在首次执行 robot.step() 即检测到 → finish_reason=admin_stop, duration_sim=0.032s 。

这就解释了为什么全部 282 条 admin_stop 记录的 duration_sim 均为 0.000 秒。

2.2 证据链

证据	说明
282 条 admin_stop 全在 0.000s	证明 STOP 文件在 Webots 启动之前就已存在
同一秒内跨团队聚集（如 00:18:38 同时停掉 race_88642a8c 和 race_ffab48e8）	批量操作在一次 API 调用中遍历所有活跃比赛
A10089 连续 12 次失败，全部 0.000s 且 sim_race_id 各不同	排除了 ID 碰撞和 STOP 文件残留假说
无团队特定模式	批量取消不区分团队

3. 代码审查发现的 Bug

Bug A（严重）： zone_reset() 不取消正在运行的比赛

文件: server/blueprints/admin.py:603-609

```
@router.post("/zones/{zone_id}/reset")
async def zone_reset(zone_id: str, _auth=Depends(require_admin)):
    sm = get_zone_sm(zone_id)
    sm.reset()                                # 只重置状态机
    _zone_running_session.pop(zone_id, None)  # 忘记 running session
    await _broadcast("idle", zone_id=zone_id)
    # ✖ 缺失: simnode_cancel_race(session_id)
    return {"status": "idle", "zone_id": zone_id}
```

危害：调用 zone_reset 后，后端认为赛区已空闲、可启动新比赛，但 simnode 上的 Webots 进程仍在运行，可能导致资源泄露和状态不一致。

修复方案：在 sm.reset() 之前添加：

```
if session_id := _get_running_session_id(zone_id):
    await asyncio.to_thread(simnode_cancel_race, session_id)
```

Bug B（中等）： graceful_stop() 的 STOP 文件清理不可靠

文件: simnode/race_runner.py:227-255

```
def graceful_stop(self, timeout: float = 15.0) -> bool:
    stop_file = self._race_dir / "STOP"
    try:
        self._race_dir.mkdir(parents=True, exist_ok=True)
        stop_file.write_text("stop", encoding="utf-8")
    except Exception as e:
        logger.warning(f"写入 STOP 信号失败 ({self.race_id}): {e}")

    if self._webots_proc is None or self._webots_proc.poll() is not None:
        return True

    try:
        self._webots_proc.wait(timeout=timeout)
        return True
    except subprocess.TimeoutExpired:
        logger.warning(f"比赛 {self.race_id}: 优雅停止超时, 强制终止")
        self.force_stop()
        return False
    finally:
        try:
            stop_file.unlink(missing_ok=True)  # ✔ 有 unlink, 但嵌套在 try 内
        except Exception:
            pass
```

问题：finally 仅在 wait() 代码块生效，如果 stop_file.write_text() 之后立即抛出异常（在其他代码路径中），unlink 不会被调用。虽然 missing_ok=True 降低了影响，但同一 sim_race_id 目录被复用时可能出现假 admin_stop。

修复方案：将 unlink 提升到外层 finally，或在 run_race() 开头添加启动清理。

Bug C (低): zone_stop_race() 无审计日志

文件: server/blueprints/admin.py:585-600

```
@router.post("/zones/{zone_id}/stop-race")
async def zone_stop_race(zone_id: str, _auth=Depends(require_admin)):
    ...
    await asyncio.to_thread(simnode_cancel_race, session_id)
    ...
    return {"status": "stopping", "zone_id": zone_id}
# ✗ 无任何日志记录调用者、IP、影响的 race_id
```

修复方案: 添加审计日志 (见第 4 节 P0 方案)。

4. 推荐修复方案

P0 — 立即执行 (止血 + 审计)

1. 在 zone_stop_race() 中添加强制审计日志 — server/blueprints/admin.py:585

```
logger.warning(
    "[AUDIT] zone_stop_race: zone=%s session=%s "
    "client=%s user=%s",
    zone_id, session_id,
    request.client.host if request else "unknown",
    credentials.username if credentials else "unknown",
)
```

2. 在 run_race() 启动时清理孤立的 STOP 文件 — simnode/race_runner.py:45

在 run_race() 方法开头, _decode_car_codes() 之前添加:

```
# 清理上一次运行可能残留的 STOP 文件
(self._race_dir / "STOP").unlink(missing_ok=True)
```

P1 — 短期 (防止复发)

3. 将 unlink() 提取到 graceful_stop() 的外层 finally — simnode/race_runner.py:229

```
def graceful_stop(self, timeout: float = 15.0) -> bool:
    stop_file = self._race_dir / "STOP"
    try:
        self._race_dir.mkdir(parents=True, exist_ok=True)
        stop_file.write_text("stop", encoding="utf-8")
        ...
    finally:
        try:
            stop_file.unlink(missing_ok=True)
        except Exception:
            pass
```

4. 修复 zone_reset() 缺失的 simnode cancel 调用 — server/blueprints/admin.py:603

```
@router.post("/zones/{zone_id}/reset")
async def zone_reset(zone_id: str, _auth=Depends(require_admin)):
    # 先取消 simnode 上正在运行的比赛
    session_id = _get_running_session_id(zone_id)
    if session_id:
        await asyncio.to_thread(simnode_cancel_race, session_id)
    # 再重置状态机
    sm = get_zone_sm(zone_id)
    sm.reset()
    _zone_running_session.pop(zone_id, None)
    await _broadcast("idle", zone_id=zone_id)
    return {"status": "idle", "zone_id": zone_id}
```

P2 — 中期（架构加固）

5. 比赛创建后添加保护窗口

在 admin.py:zone_start_race 中创建比赛后，为比赛设置一个 5 秒的"不可取消"窗口。cancel_race() 在此窗口内返回 HTTP 423 Locked（除非 force=true）。

6. 所有取消端点添加调用者元数据到日志

P3 — 长期（消除根因类别）

7. 用 IPC 机制替代文件系统的 STOP 信号

- 可选方案：
- (a) Webots supervisor customData 通道
 - (b) multiprocessing.Event 或共享内存标志
 - (c) supervisor 轮询的 localhost HTTP 端点
- 以上任一方案永久消除 STOP 文件残留、竞态条件、清理 bug 这一整个错误类别。

5. 服务器端待排查项（必须在服务器上执行）

优先级	检查项	命令
P0	查询谁调用了取消 API	grep "stop-race cancel" /var/log/nginx/access.log grep "00:1[5-8]"
P0	查询 FastAPI 日志中的 zone_stop_race	grep "zone_stop_race stop-race" /path/to/app.log
P1	检查 STOP 文件是否残留在磁盘	find /home/pkudsa/main_branch/recordings/ -name "STOP" -ls
P1	查看 supervisor 调试日志	cat .../race_ffab48e8/_debug_supervisor.log
P2	检查 cron/systemd 定时任务	crontab -l && systemctl list-timers && ls /etc/cron.*
P3	simnode 日志中的取消事件	grep "cancel_race graceful_stop STOP" /path/to/simnode.log

优先级	检查项	命令
P4	验证 sim_race_id 无碰撞	SELECT session_id, COUNT(*) FROM race_sessions GROUP BY session_id HAVING COUNT(*) > 1

6. 关键代码位置速查

文件	行号	作用
server/blueprints/admin.py	585-600	zone_stop_race — 批量取消入口，无审计日志
server/blueprints/admin.py	603-609	zone_reset — 缺少 simnode cancel 调用
server/utils/simnode_client.py	86-94	cancel_race() HTTP 客户端 → simnode
simnode/server.py	137-145	POST /race/{race_id}/cancel REST 端点
simnode/race_manager.py	141-170	cancel_race() — 协调优雅停止
simnode/race_runner.py	229-255	graceful_stop() — 写入 STOP 文件
simnode/race_runner.py	45	run_race() 入口 — 缺少孤儿 STOP 清理
simnode/webots/controllers/supervisor/supervisor.py	542-545	STOP 检测 → admin_stop
server/services/test_worker.py	301	sim_race_id 生成 — race_{race_id[:8]}

7. 结论

根因已确认： 大量 admin_stop 由对 POST /api/admin/zones/{zone_id}/stop-race 的批量调用造成。由于 graceful_stop() 写入 STOP 文件的速度远快于 Webots 启动速度，在竞态条件下所有比赛在第 1 帧即被终止，duration_sim = 0.000s。

"谁调用了批量取消" 是本机上无法回答的最后一个问题——需要通过服务器上的 HTTP 访问日志来确认具体的调用者。但**代码修复可以不依赖此信息先行进行**，以上 P0-P1 方案在根因确认之前就应该部署以防止继续发生。